Enum, Iterator, Protocol, Comprehension, Multiple Inheritance

COMP3522 OBJECT ORIENTED PROGRAMMING 2

WEEK 5

Enums

I LOVE ENUMS!

Enums

Coding problem

Imagine I ask you to write a calendar program

You might represent the days in a week by the numbers 0-6, with 0 meaning Sunday -> 6 meaning Saturday

However if we write something like

- `day = 0`

It's a little difficult to realize that 0 in this case means Sunday

Enums

A slightly better approach is to define a series of **constants** to represent each day, then later on we assign the constants to the day

```
SUNDAY = 0
MONDAY = 1
…
SATURDAY = 6
```

Then we write something like

- `day = SUNDAY`

The code is definitely easier to read, but we can do better.

Enums

There are only 7 days in a week, it'd be great if we could wrap the 7 day constants in some data type

An enumeration (**Enum**) is a user-defined type that consists of a set of **named value constants** that are known as **enumerators**

The members of an enumeration can be compared by these names, and the enumeration itself can be iterated over

Whenever we need to **represent simple constant data over a finite range**, Enums are a great option

- Makes code easier to use and read!
- What looks easier to understand?
  - `if day == 1:`
  - `if day == Days.MONDAY:`

Enums

```
import enum
class Days(enum.Enum):
  SUN = 0
  MON = 1
  TUE = 2
  WED = 3
  THU = 4
  FRI = 5
  SAT = 6
```

Let's represent the previous days of the week example using enums

Enums in Python are classes that inherit from the Enum class

Make sure to import from the **enum module**

Variables that are set in the enum are the enumerators

- Effectively setting constants to a specific value

Enums

```
import enum
class Days(enum.Enum):
  SUN = 0
  MON = 1
  TUE = 2
  WED = 3
  THU = 4
  FRI = 5
  SAT = 6
```

Enums can be accessed through their name or value.

Notice how there's no need to instantiate the Days class:

```
 d = Days(3)
```

```
day = Days.MON
print(type(day)) #<enum 'Days'>
print(day) #Days.MON
print(day.name) #MON
print(day.value) #1
print (Days['SAT']) #Days.SAT
print (Days(4)) #Days.THU
```

Enums

```
import enum
class Days(enum.Enum):
  SUN = 0
  MON = 1
  TUE = 2
  WED = 3
  THU = 4
  FRI = 5
  SAT = 6
```

Can not modify the name or value of an enum after it's been created

```
day = Days.MON
day.name = Days.THU.name #ERROR
day.value = 5 #ERROR

day = Days.TUE #OK! Changing day to Days.TUE
```

Enums

```
import enum
class Days(enum.Enum):
  SUN = 0
  MON = 1
  TUE = 2
  WED = 3
  THU = 4
  FRI = 5
  SAT = 6
```

Can print out all enum members as though the Enum class is an iterable object

```
for day in Days:
  print(day)
```

Output:
Days.SUN
Days.MON
Days.TUE
Days.WED
Days.THU
Days.FRI
Days.SAT

Enums

```
import enum
class Days(enum.Enum):
  SUN = 0
  MON = 1
  TUE = 2
  WED = 3
  THU = 4
  FRI = 5
  SAT = 6
  SAT = 9 #ERROR
  SAT2 = 6 #OK
```

Can NOT have member within an **enum have the same name**

BUT

You can have different members within an **enum that have the same value**

Real world example: Representing clothing in a videogame

You need to have a finite number of clothing options in your game

Each piece of clothing needs to have a unique id

There must be a way to find the number of:

- Only shoes
- Only pants
- All items

How can this be achieved with only enums?

https://www.youtube.com/watch?v=01Gf3Oqfb3Q&t=420s

Real world example: Representing clothing in a videogame

```python
import enum
from enum import auto
class ClothingId(enum.Enum):

    SHOE_1 = auto() #1
    SHOE_2 = auto() #2
    SHOE_3 = auto() #3
    PANT_1 = auto() #4
    PANT_2 = auto() #5
    PANT_3 = auto() #6
    PANT_4 = auto() #7
    PANT_5 = auto() #8
```

auto() is a built-in python function that automatically assigns values to enum members.

Values start at 1 by default

All clothing items have a unique id

But there's no way to:

- Get the total number of unique clothes
- Get only the shoes
- Get only the pants

Real world example: Representing clothing in a videogame

```
import enum
from enum import auto

class ClothingId(enum.Enum):
    SHOE_BEGIN = SHOE_1 = auto() #1
    SHOE_2 = auto() #2
    SHOE_3 = auto() #3
    SHOE_END = PANT_BEGIN = PANT_1 = auto() #4
    PANT_2 = auto() #5
    PANT_3 = auto() #6
    PANT_4 = auto() #7
    PANT_5 = auto() #8
    PANT_END = NUM_CLOTHES = auto() #9
```

Create "**Bookmarks**" in the enum list

- **SHOE_BEGIN**, **SHOE_END**, **PANT_BEGIN**, **PANT_END**, **NUM_CLOTHES** indicate special points in the enum list
- Most have the same value as an existing enum

Use these "**Bookmarks**" as points to separate shoes and pants

We can now:

- Get the total number of unique clothes
- Get only the shoes
- Get only the pants

clothing_enum.py

Iterators

What’s an iterator (the iter function)

Python and other languages support the concept of **iterating** over containers:

```
my_tuple = ("apple", "banana", "cherry")
my_it = iter(my_tuple)
print(my_it.__next__()) #apple
print(next(my_it)) #banana
print(next(my_it)) #cherry
```

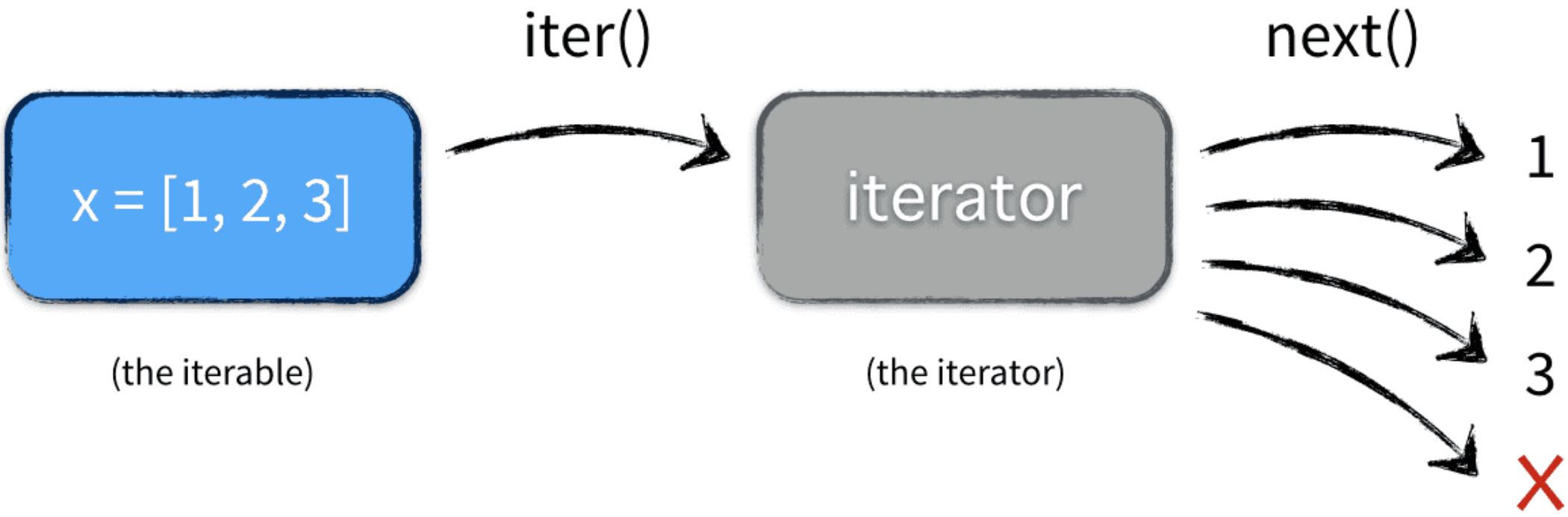
An **iterator is an object that contains a countable number of values**

We can repeatedly call the iterator’s **next method** or use the next global function

We can iterate upon an iterator

We can traverse all the values

Iterator and Iterable



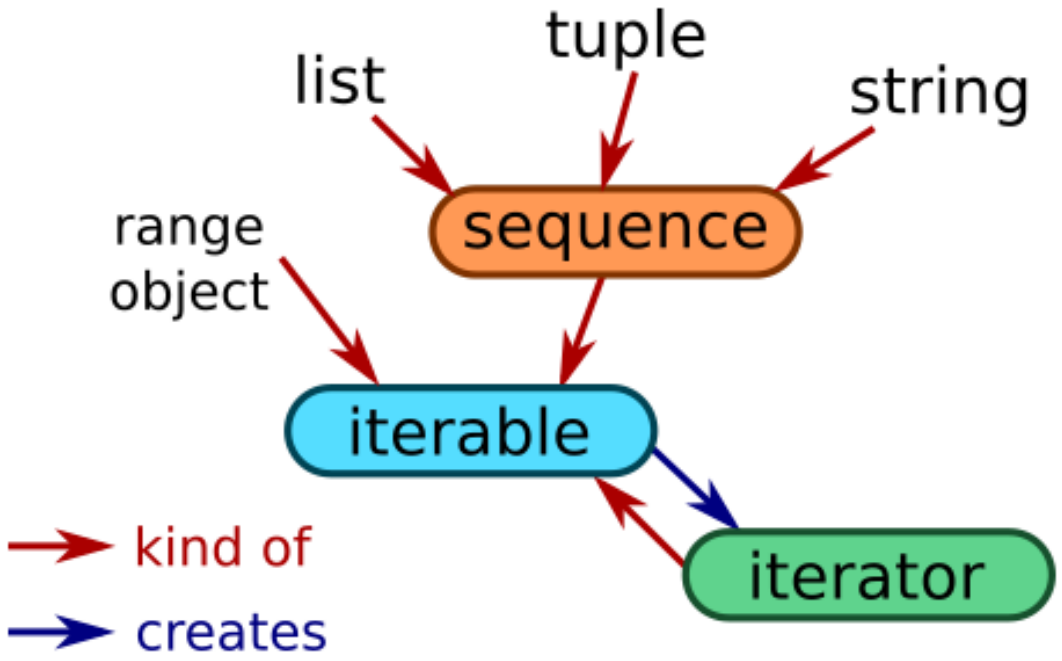
Iterator vs Iterables

An **Iterator** is responsible for traversing an **Iterable**

An **iterators** primary responsibility is to **iterate** over the elements of an **iterable.**

An Iterable is something that you can loop over.

- List
- Dictionary
- Tuples
- Sets
- Any container or sequence really
- Custom data types



How does an iterator work?

It **streams** the contents of the container

Calling **next( )** returns successive items in the stream

When no more data is available, a **StopIteration** exception is raised

When this happens, the iterator is exhausted and cannot be used again

A container object like a list produces a fresh iterator each time we:

1. Pass it to the iter( ) function
2. Use it in a **for loop**.

Iterator use examples

Strings are also iterable:

```
my_str = "banana"
my_it = iter(my_str) #explicit iterator
print(next(my_it)) #b
print(next(my_it)) #a
print(next(my_it)) #n
print(next(my_it)) #a
print(next(my_it)) #n
print(next(my_it)) #a
print(next(my_it)) #StopIteration exception
```

Iterator use examples

Strings are also iterable: **Using while loop WITH explicit iterator**

```
my_str = "banana"
my_it = iter(my_str) #explicit iterator

while True:
    try:
        item = next(my_it)
    except StopIteration:
        break

    print(item)
```

Iterator use examples

Strings are also iterable: **Using for loop WITH explicit iterator**

```
my_str = "banana"
my_it = iter(my_str) #explicit iterator

for item in my_it:
    print(item)
```

Iterator use examples

Strings are also iterable: **Using for loop WITHOUT explicit iterator**

```
my_str = "banana"

for item in my_str: #hidden iterator auto generated
    print(item)
```

Iterator use examples

```
animal_and_foods = {
    'dog': 'bone',
    'cat': 'fish',
    'bird': 'seed',
    'elephant': 'peanuts'
}

iterator = iter(animal_and_foods)
print(next(iterator))
print(next(iterator))
print(next(iterator))
print(next(iterator))
```

Getting iterator from dictionary retrieves the keys

Output:

dog
cat
bird
elephant

Iterator use examples

```
animal_and_foods = {
    'dog': 'bone',
    'cat': 'fish',
    'bird': 'seed',
    'elephant': 'peanuts'
}

iterator = iter(animal_and_foods.values())
print(next(iterator))
print(next(iterator))
print(next(iterator))
print(next(iterator))
```

Output:

bone
fish
seed
peanuts

Need to explicitly call **values** method to get iterator of values

Iterator use examples

```
animal_and_foods = {
    'dog': 'bone',
    'cat': 'fish',
    'bird': 'seed',
    'elephant': 'peanuts'
}

iterator = iter(animal_and_foods.items())
print(next(iterator))
print(next(iterator))
print(next(iterator))
print(next(iterator))
```

Need to explicitly call **items** method to get iterator of key/value pairs

Output:

('dog', 'bone')
('cat', 'fish')
('bird', 'seed')
('elephant', 'peanuts')

PROTOCOLS

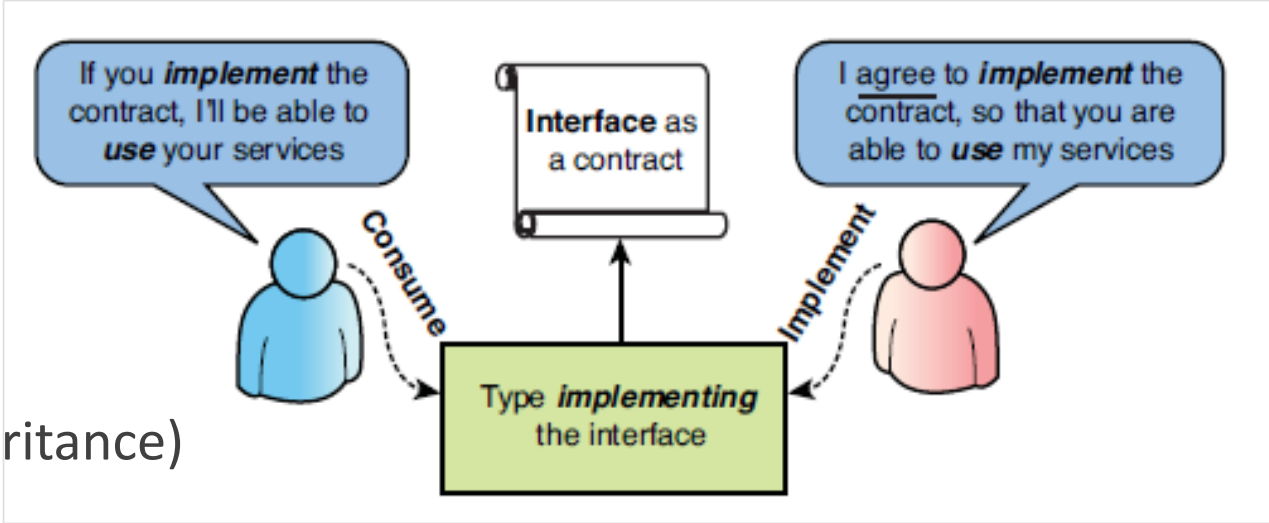
Remember Interfaces? (I'm going to keep bringing this up)

An Interface is a contract demanding that the class implement certain methods and behaviors.

In more general OOP terms, an interface refers to the public methods and attributes that are provided by a class. An interface determines how we can interact with an object. These public attributes and methods are the only way to communicate (or interface) with the class.

In Python there are two kinds:

1. Informal interfaces
   a) Duck typing
   b) **Protocols**
2. Formal interfaces
   a) Abstract base classes (using inheritance)



The Python protocol

Protocols are collections of methods that Python developers and libraries expect certain “kinds” of things to have

This is analogous to interfaces in the Java Collections Framework

There are some very helpful protocols in Python

In order to implement a protocol, a type must support certain operations

The Sized protocol

Implemented by having a method called **__len__(self)**

Invoked when an instance is passed to the built-in function len( )

This is how an object reports the correct value to the len( ) function

We may assume Python's built-in len( ) function works something like this:

```
def len(obj):
    return obj.__len__( )
```

__len__(self) returns the length of the object, an integer >= 0

Implemented by str, list, range, tuple, bytes, set,dict

team_sized.py

The Container protocol

Provides the method **__contains__(self, item)**

When the membership operator **in** or **not in** is invoked on an object, this is the method invoked

This is how a class implements the membership test operator

__contains__(self, item) must **return true** if the item is **in** self, **else false**

Implemented by str, list, range, tuple, bytes, set,dict

team_container.py

The Iterable protocol

Provides the method **__iter__(self)**

Invoked to generate an iterator object for a container

Should **return a new iterator** object that can iterate over all the objects in the container

Invoked by

1. passing the object to the global Python `iter( )` function
2. using the for-loop, i.e., `for item in iterable`

team_iterable.py

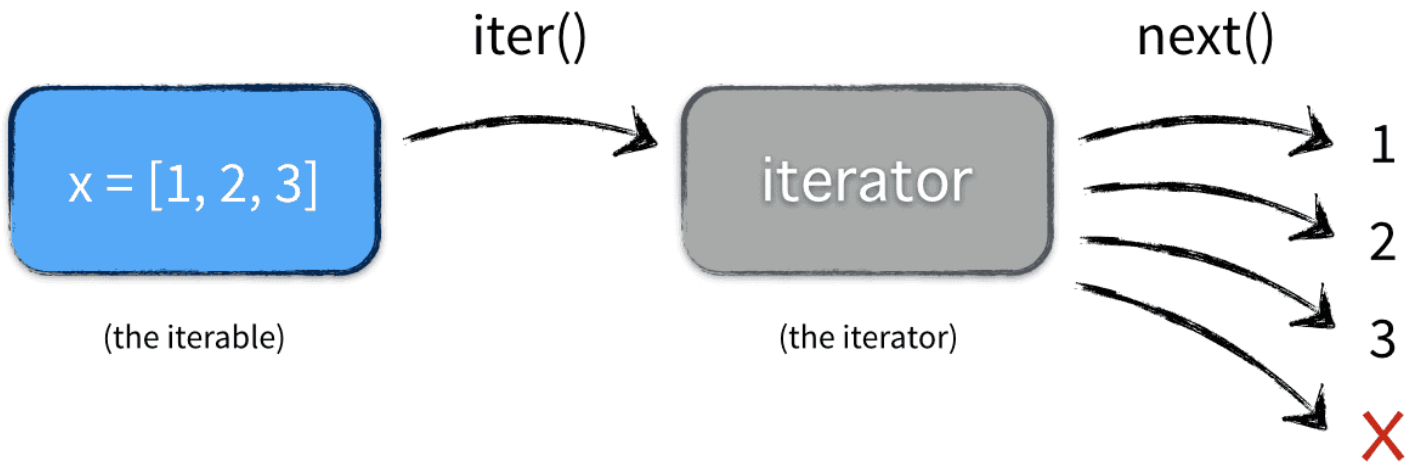
How would we implement this from scratch in Python? (The Iterator Protocol)

Iterator:

- Implements `__next__(self)` #returns next item
- Implements `__iter__(self)` #returns itself

Iterable:

- Implements `__iter__(self)` #returns an iterator



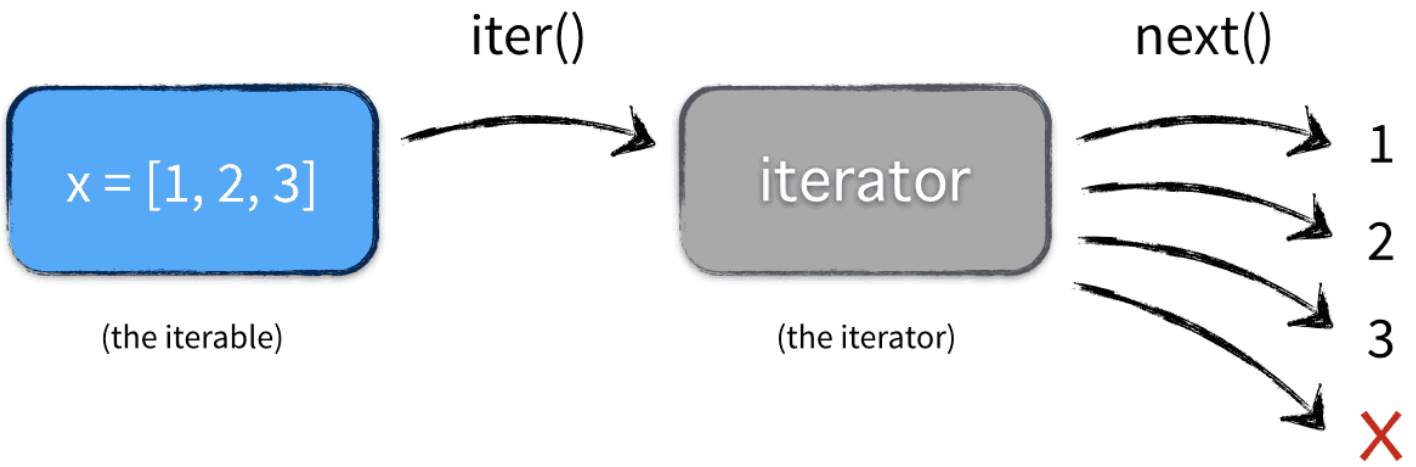
How would we implement this from scratch in Python? (The Iterator Protocol)

Say we wanted to iterate over each word of a sentence and print out the word with its letters CAPITALIZED.

How would we implement this in python?

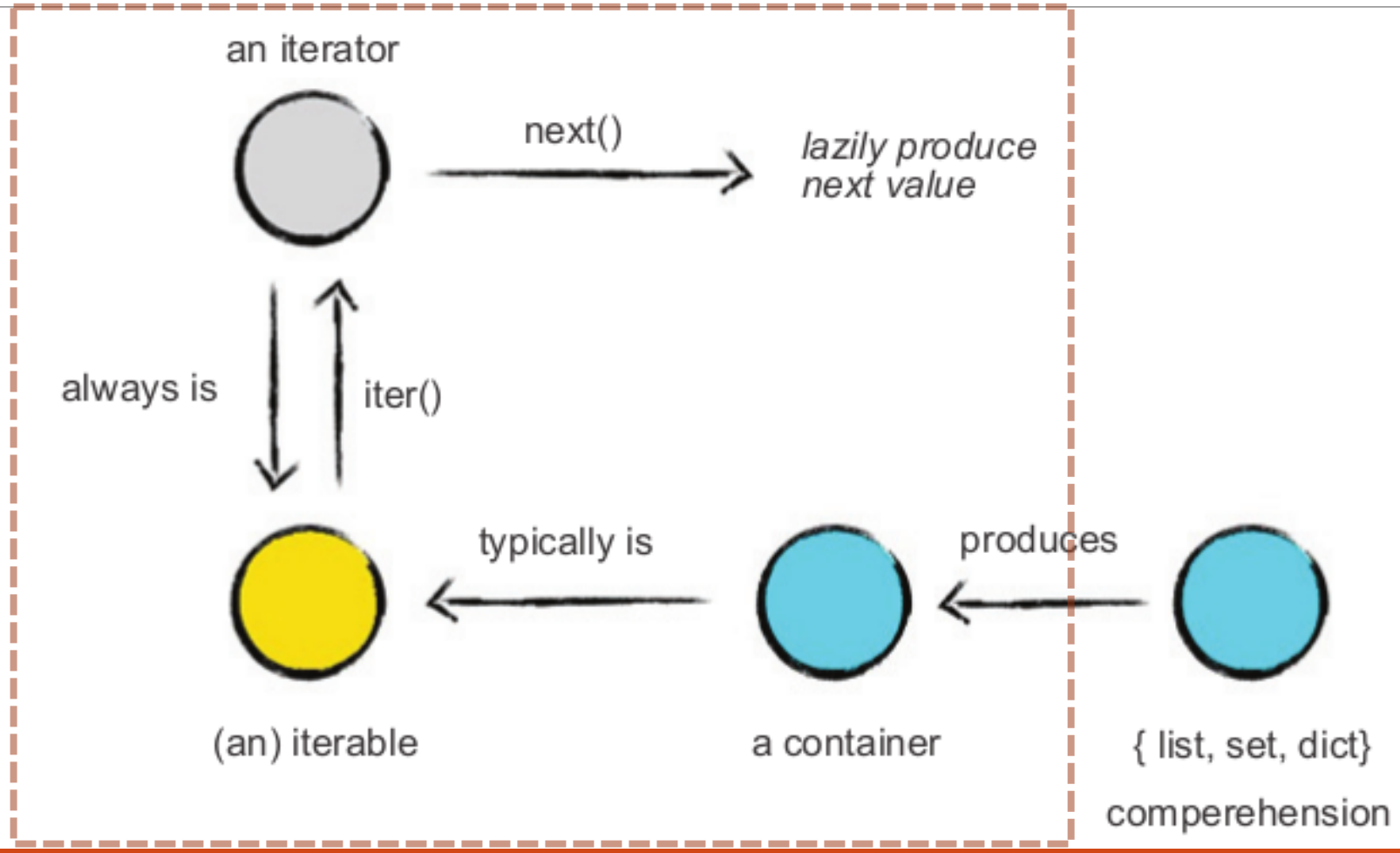
We would need to implement an **iterable class** and an **iterator class**.

Let's look at some code: iterator_iterable.py

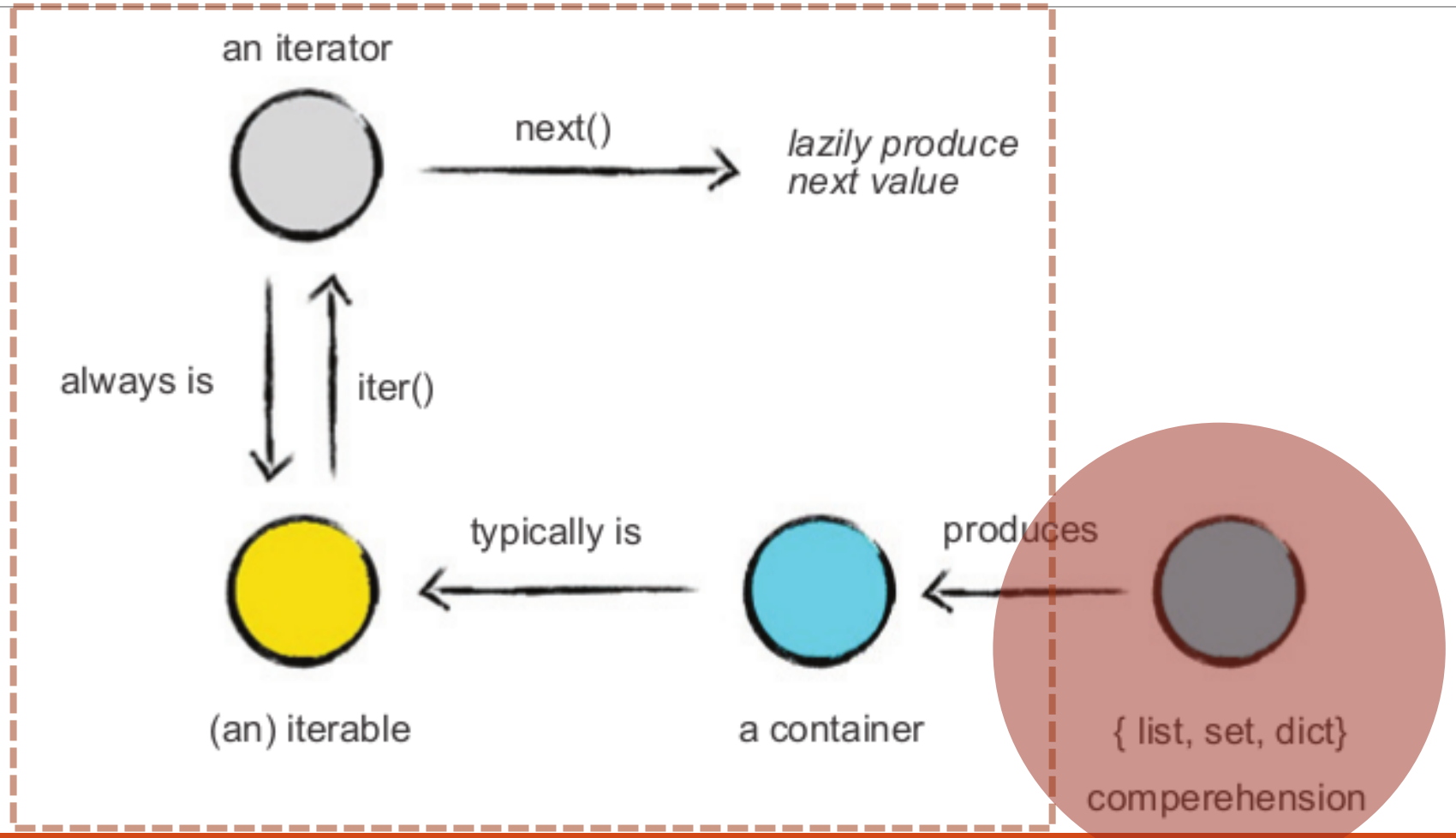


Comprehensions

So this is what we did in the code we just saw



So this is what we did in the code we just saw



What on earth are these?

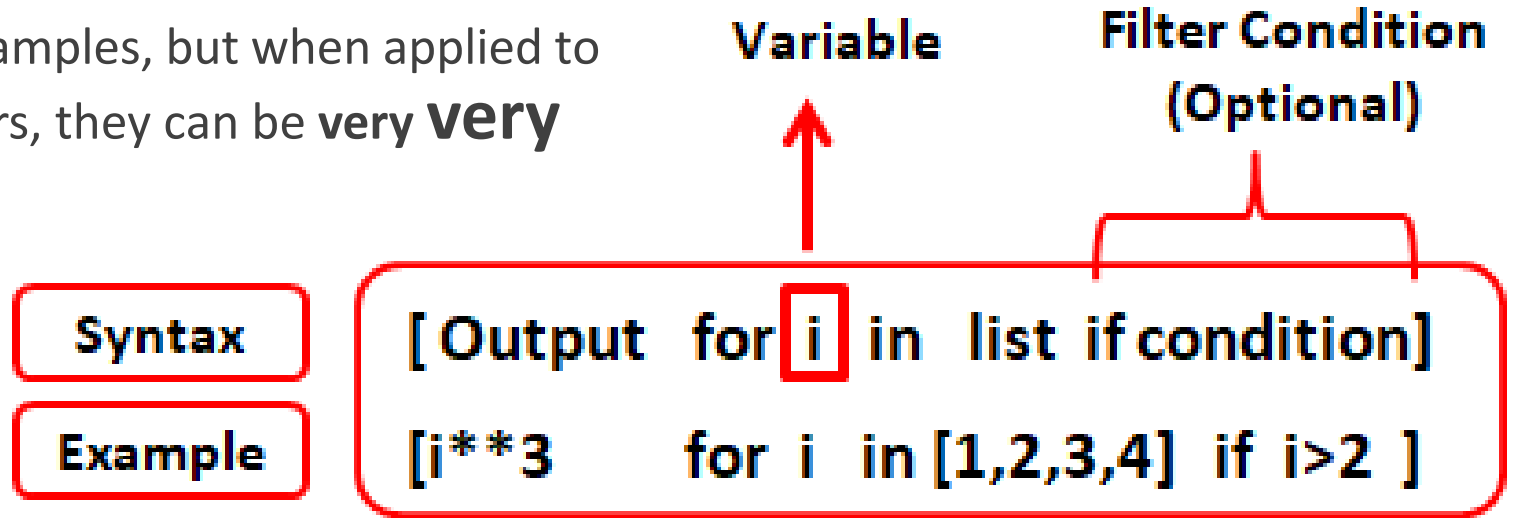
Comprehensions

Comprehensions allow us to transform/filter an iterable object into another iterable.

In simple words, **create one iterable from another iterable** (sequence/container) by **following some special rules/filters**.

Simple, readable syntax

May seem irrelevant in simple examples, but when applied to complex sequences and containers, they can be **very** **very powerful**.



Comprehensions

Problem: Write code that takes a range of numbers 0-10 (excluding 10) and generates a list where the odd numbers are replaced with 'x'.

```
list = []
for x in range(0,10):
    if x % 2 == 0:
        list.append(x)
    else:
        list.append('x')
```

That's a lot of code to perform a relatively simple task. **Comprehensions** allow us to compress those 6 lines of code into one line of code

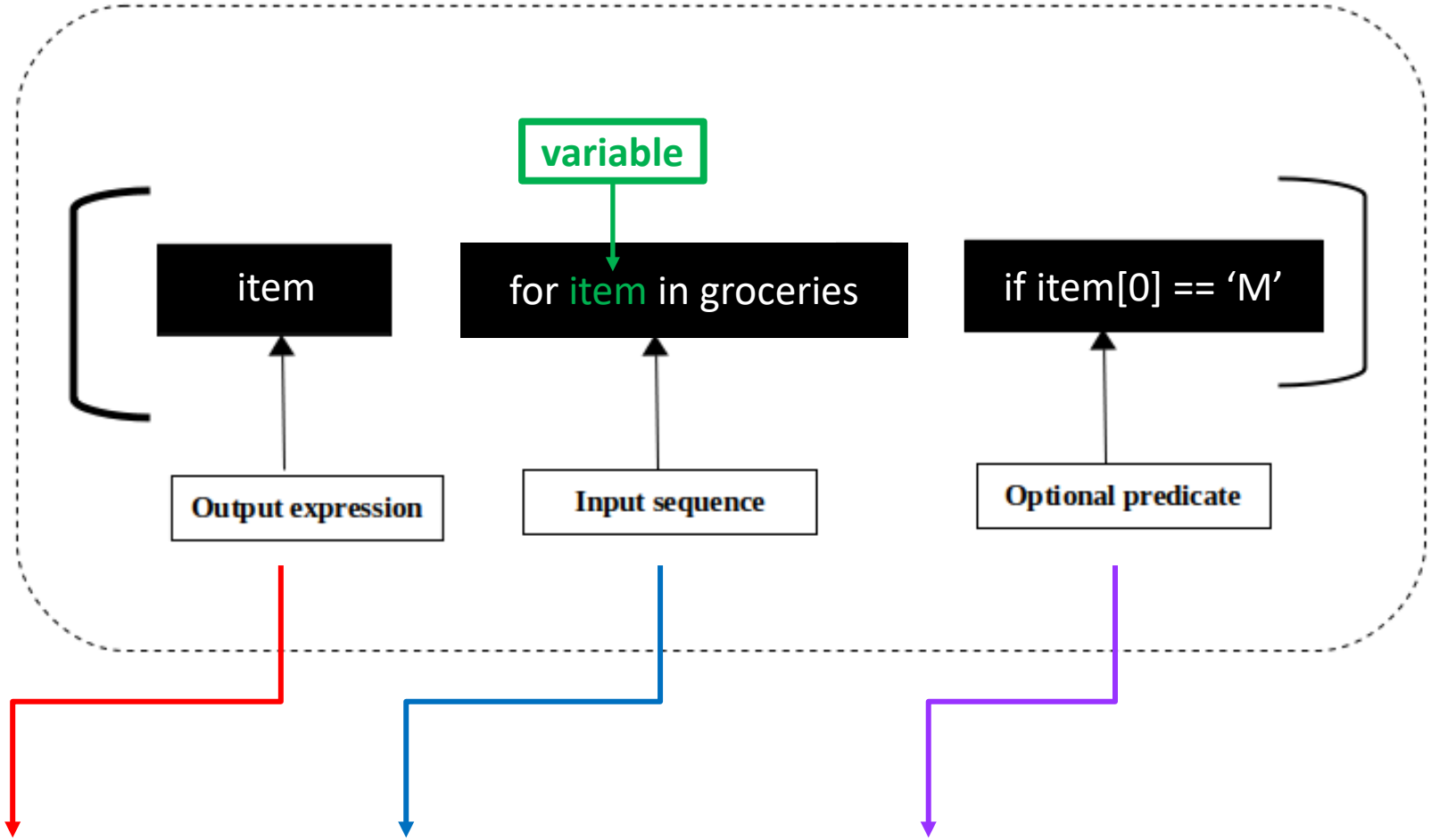
```
#same result using list comprehension
list = [x if x % 2 == 0 else 'x' for x in range(0,10)]
```

Syntax

Say we have the following iterable:

groceries = [“Bread”, “Milk”, “Mangoes”, “Mandarins”, “Onions”]

We could create a new list of all the elements starting with ‘M’:



groceries_m = [item for item in groceries if item[0] == ‘M’]

Comprehension challenges

Challenge Time

Challenge 1:

Write a list comprehension that takes a list of strings and generates a list of strings

```
strings = ['hi','hello','bye’]
new_strings = ???
```

Challenge Time

Challenge 1:

Write a list comprehension that takes a list of strings and generates a list of strings

```
strings = ['hi','hello','bye’]
new_strings = [s for s in strings]
```

Challenge Time

Challenge 2:

Write a list comprehension that takes in numbers in the range of 026 (excluding 26) and generates all the letters in the alphabet (uppercase)

HINT: chr(integer) converts an int to a character. Ie: chr(65) is A

```
alphabet = ???
```

Challenge Time

Challenge 2:

Write a list comprehension that takes in numbers in the range of 026 (excluding 26) and generates the all the letters in the alphabet (uppercase)

HINT: chr(integer) converts an int to a character. Ie: chr(65) is A

```
alphabet = [chr(x+65) for x in range(0,26)]
```

Challenge Time

Challenge 3:

Write a list comprehension that takes a range of numbers 0-10 (excluding 10) and generates a list of all the even numbers in that range

```
even = ???
```

Challenge Time

Challenge 3:

Write a list comprehension that takes a range of numbers 0-10 (excluding 10) and generates a list of all the even numbers in that range

```
even = [x for x in range(0,10) if x % 2 == 0]
```

Challenge Time

Challenge 4:

Write a single list comprehension that takes a range of numbers 0-10 (excluding 10) and generates a list that contains all the same numbers, except that the odd numbers are now replaced with ‘x’.

The syntax changes if you use an **if-else**:

[f(x) if condition else g(x) for x in some_iterable]

Red – Output, Purple – Conditional, Blue – Original sequence

```
replaced = ???
```

Challenge Time

Challenge 4:

Write a single list comprehension that takes a range of numbers 0-10 (excluding 10) and generates a list that contains all the same numbers, except that the odd numbers are now replaced with ‘x’.

The syntax changes if you use an **if-else**:

[f(x) if condition else g(x) for x in some_iterable]

Red – Output, Purple – Conditional, Blue – Original sequence

```
replaced = [x if x % 2 == 0 else ‘x’ for x in range(0,10)]
```

Dictionary comprehensions, summary

Dictionary comprehension

Exactly the same as a list comprehension, but our output expression is now a key- value pair.

Eg: Double each value in a **dictionary**:

my_dictionary = {‘a’: 1, ‘b’: 2, ‘c’:3, ‘d’:4}

dictionary_2 = {key : value*2 for key,value in my_dictionary.**items()**}

Output: {‘a’: 2, ‘b’: 4, ‘c’ :6, ‘d’:8}

{ f(key): g(value) for key,value in some_iterable if conditional }

Red – Output, Blue – Original iterable, Purple - condition

Dictionary comprehension

We can even make dictionaries from lists, ranges and other iterables!

{ chr(i + 65) : i + 65 for i in range(0,26) }

Red – Output, Blue – Original iterable, Purple – filter condition (optional)

How we might read the expression is:

Create key (character), value (int) pairs in the range 0-26 and store it in a dictionary {}

{ ‘A’: 65, ‘B’:66, ’C’:67, ….. , ‘Z’:90 }

Why should we use comprehensions?

What's the point of Comprehensions?

- Avoid messy while and for loops. Which one looks easier to you?

```
#using for loop
list = []
for x in range(0,10):
    if x % 2 == 0:
        list.append(x)
    else:
        list.append('x')
```

```
#same result using list comprehension
list = [x if x % 2 == 0 else 'x' for x in range(0,10)]
```

What’s the point of Comprehensions?

- Comprehensions hide the working of iterators
- Comprehensions leverage the power of iterators to perform complex operations.
- Comprehensions are faster
- Comprehensions are readable
- Comprehensions are maintainable

Comprehensions with more complex data.

Let's see an example where we parse the following log file snipper with Comprehensions.

```
user@host> file show /var/log/processes Feb 22 08:58:24 router1 snmpd[359]:
%DAEMON-3-SNMPD_TRAP_WARM_START: trap_generate_warm: SNMP trap: [Warning]
warm start
Feb 22 20:35:07 router1 snmpd[359]:
%DAEMON-6-SNMPD_THROTTLE_QUEUE_DRAINED: [Warning]
trap_throttle_timer_handler: cleared all throttled traps
Feb 23 07:34:56 router1 snmpd[359]:
%DAEMON-3-SNMPD_TRAP_WARM_START: trap_generate_warm: SNMP trap: warm start
Feb 23 07:38:19 router1 snmpd[359]:
%DAEMON-2-SNMPD_TRAP_COLD_START: trap_generate_cold: [Warning] SNMP trap:
cold start
```

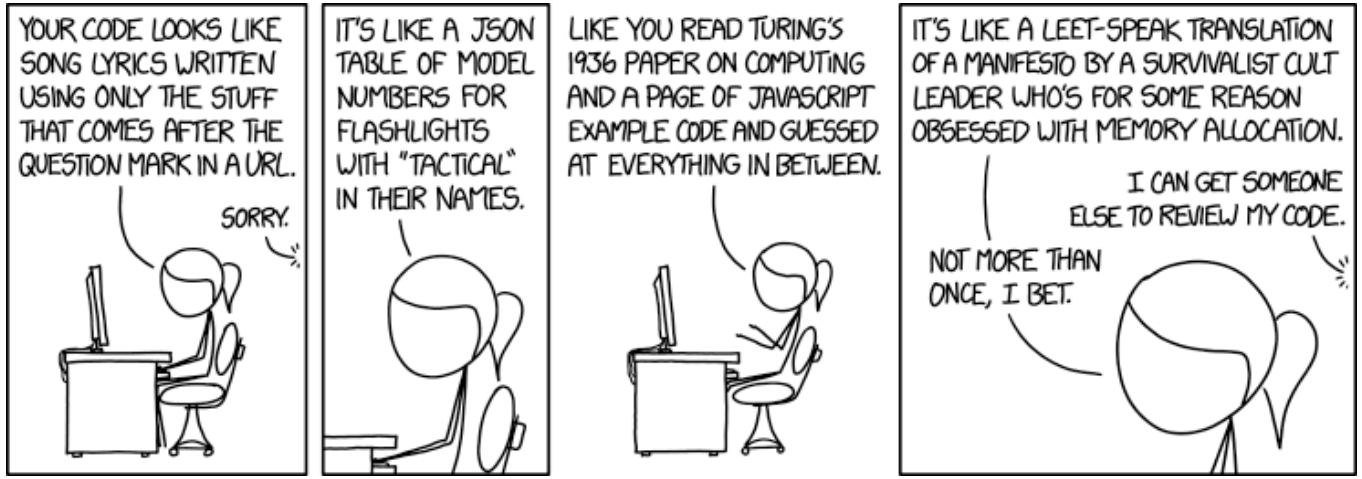
file_comprehension.py

Abusing Comprehensions

<u>Our goal as developers is to write maintainable code.</u>

Do not write comprehensions, if the expression is so convoluted that it's not readable and maintainable.

Unless we are saving a lot of space and gaining tons of speed, it's ok to switch to a for loop or while loop **<u>if</u>** it is more readable

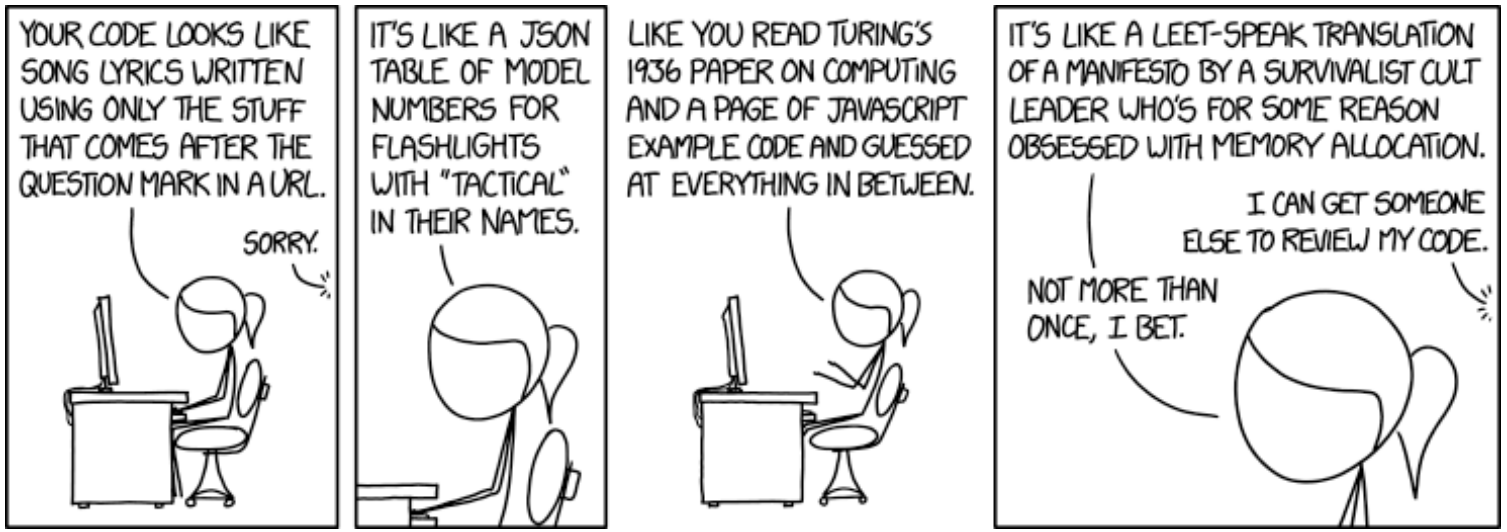


Abusing Comprehensions

That being said comprehensions are the preferred pythonic way of operating oniterables.

And there is the fact that **programmers like writing code that's concise and feels magical.**

(Except for the person debugging your code)



Abusing Comprehensions

And if the previous reasons weren't enough, if you do the following in python:

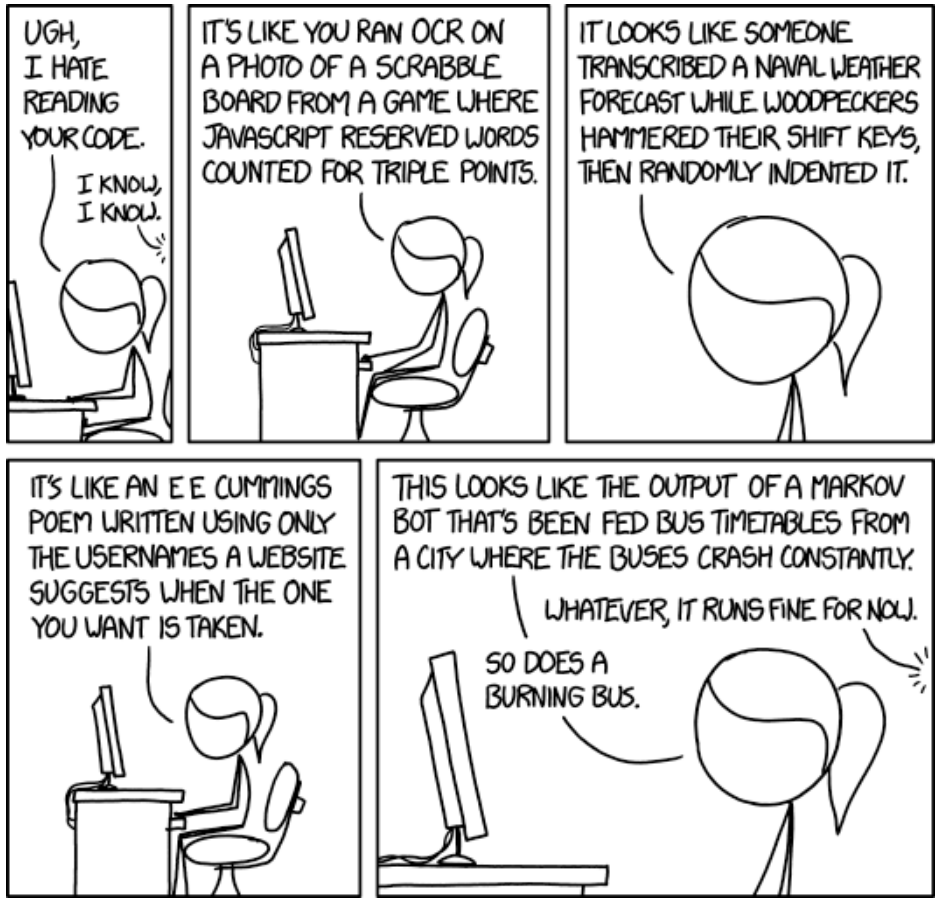
- Use tons of for loops to carry out operations on iterables

- Try using for loops traditionally with range(num_iteration)

- Don't use protocols

- Put each class in its own module all the time.

Then you **might** get a reaction like this from other python developers.



Multiple Inheritance

COMP3522 OBJECT ORIENTED PROGRAMMING 2

WEEK 5 PART 2

Rule of Thumb

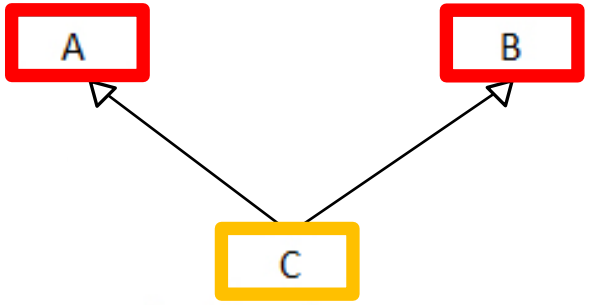
As a rule of thumb, if you think you need multiple inheritance, you're probably wrong, but if you know you need it, you're probably right.

- Dusty Philips, Python3 Object-oriented Programming

Multiple Inheritance

A VERY sensitive subject.

When one **child class** inherits from one or more **base classes** and can use functionality from all the base classes.



Multiple Inheritance

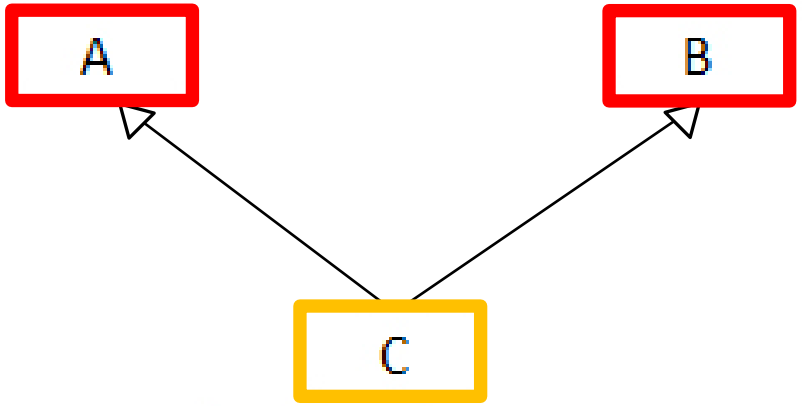
My Sagely Advice:

Avoid it if you can, there are alternatives and they are usually simpler to implement.

Many languages don't support multiple Inheritance.

That being said, it is a feature and if it is needed, use it RESPONSIBLY. (Keep it simple and minimal)

Multiple Inheritance



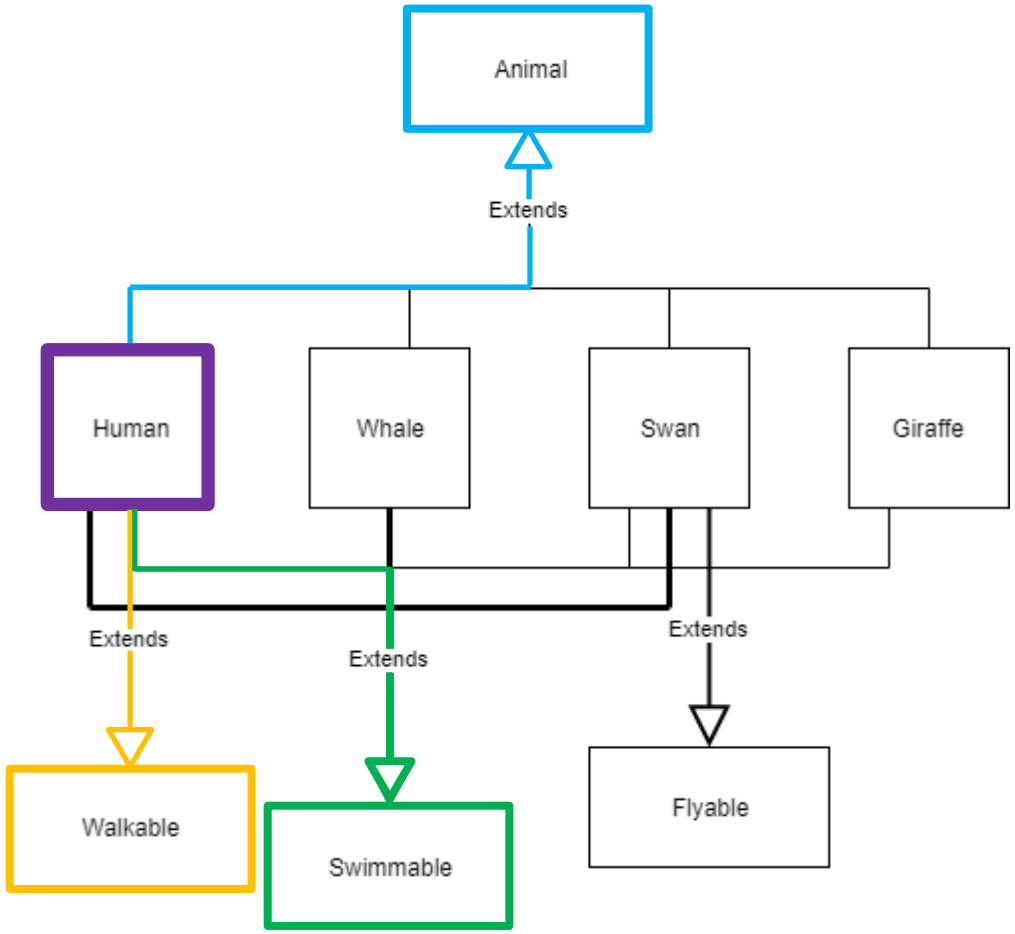
Multiple Inheritance

```
class C(A, B):
    # optional overridden functionality of A
    # optional overridden functionality of B
    # Implementation of C
    pass
```

All the rules of inheritance apply here.

Multiple inheritance is usually the tool of choice to use when implementing multiple interfaces (Remember the **Interface Segregation Principle**?)

RECAP: Interface Segregation Principle



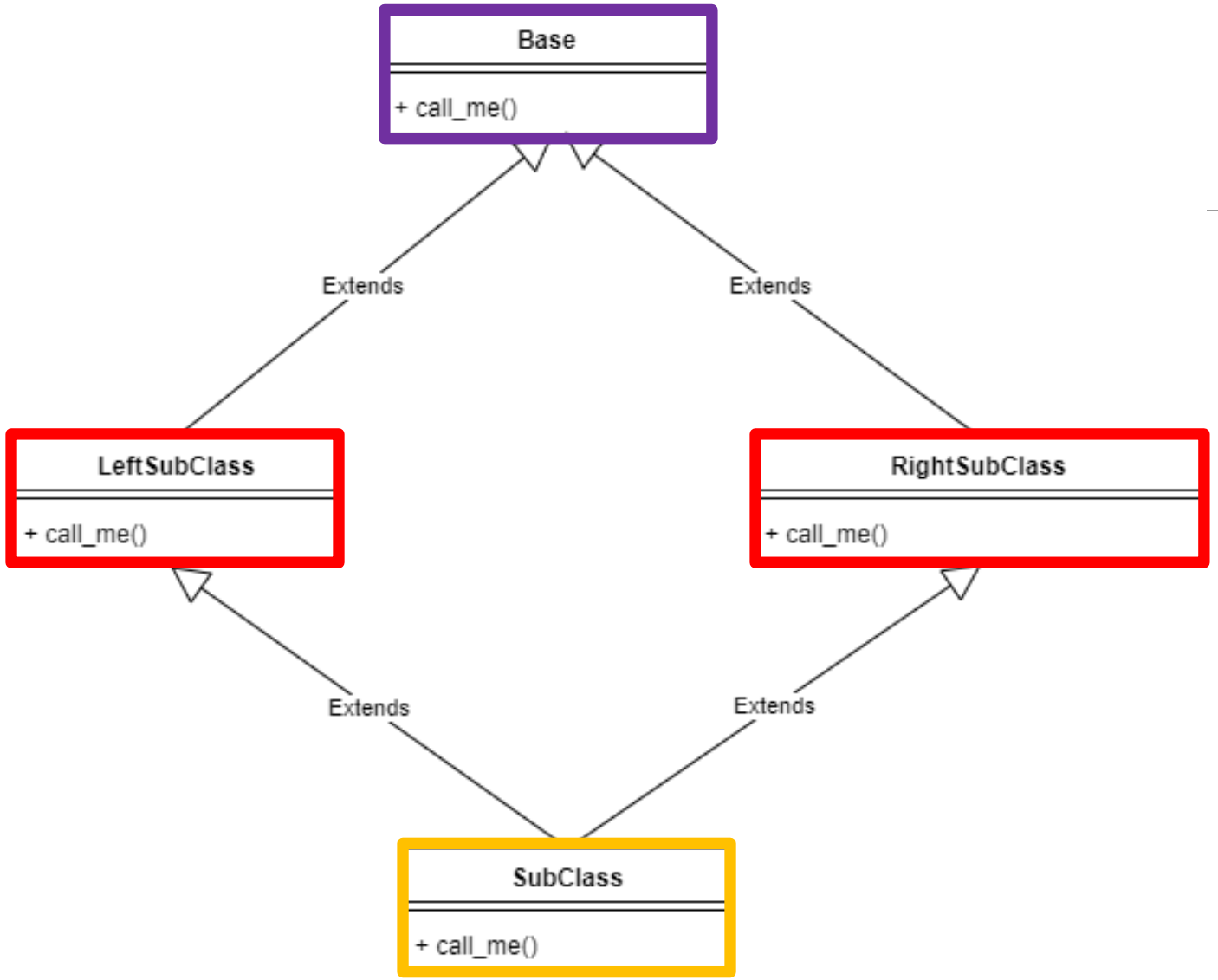

```
class Walkable(abc.ABC):
    @abc.abstractmethod
    def walk(self):
        pass
class Swimmable(abc.ABC):
    @abc.abstractmethod
    def swim(self):
        pass
class Human(Animal, Walkable, Swimmable):
    def walk(self):
        # overridden walking code
    def swim(self):
        # overridden swimming code
```

Instead of polluting the animal base class with additional functionality, we create separate interfaces (read: ABC) that handle different responsibilities.

This may seem difficult to maintain, but in fact it isn't. **This is only possible if the interfaces are well defined and have no overlap.**

The Diamond Problem

A SIMPLE TAKE AT WHY MULTIPLE INHERITANCE IS PROBLEMATIC

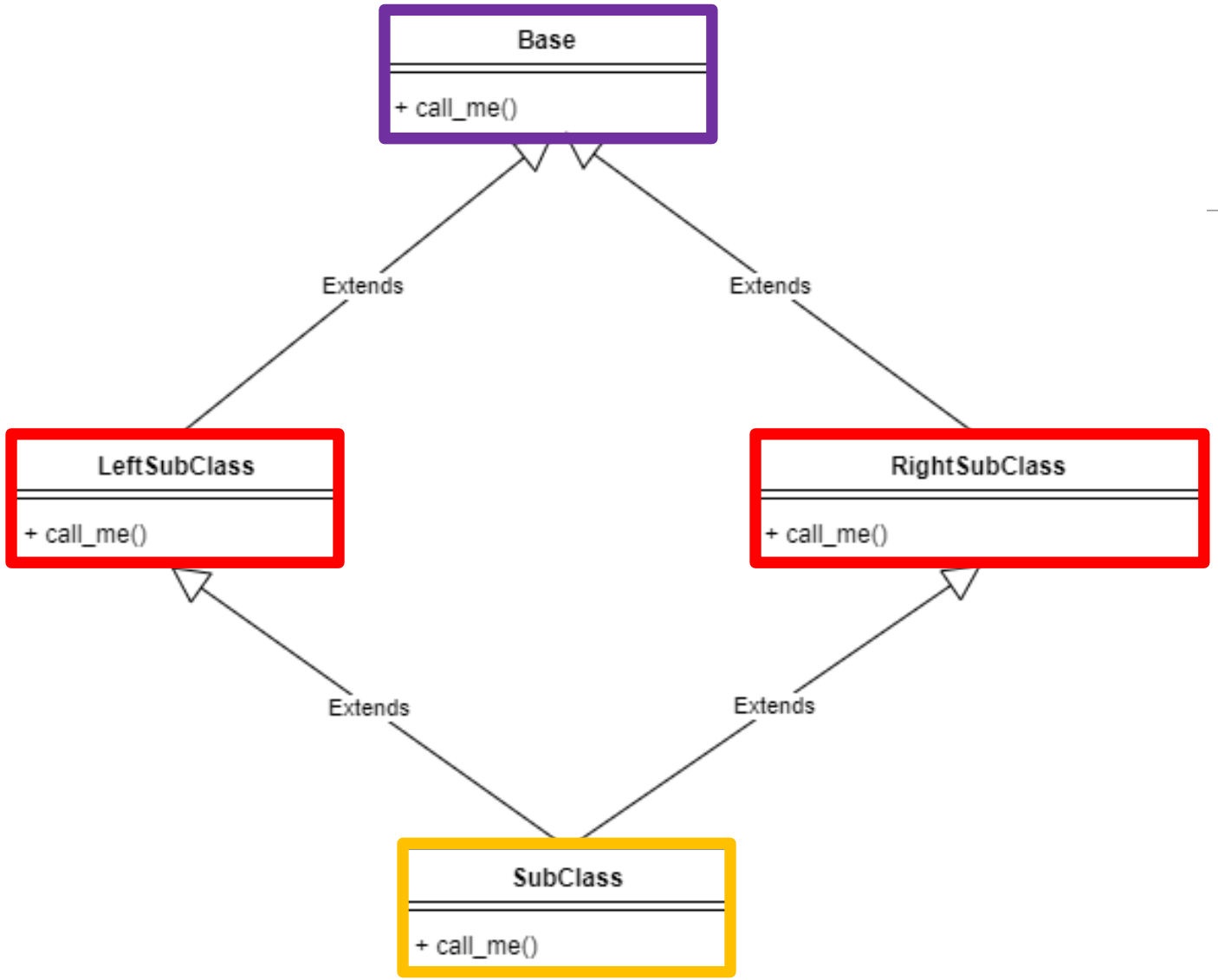


The Diamond Problem

At first glance this inheritance looks fine

Subclass inherits from 2 **parents** (**LeftSubClass** and **RightSubClass**). And **Subclass** also has a grandparent **Base**.

But if we look a little closer, some questions start to rise

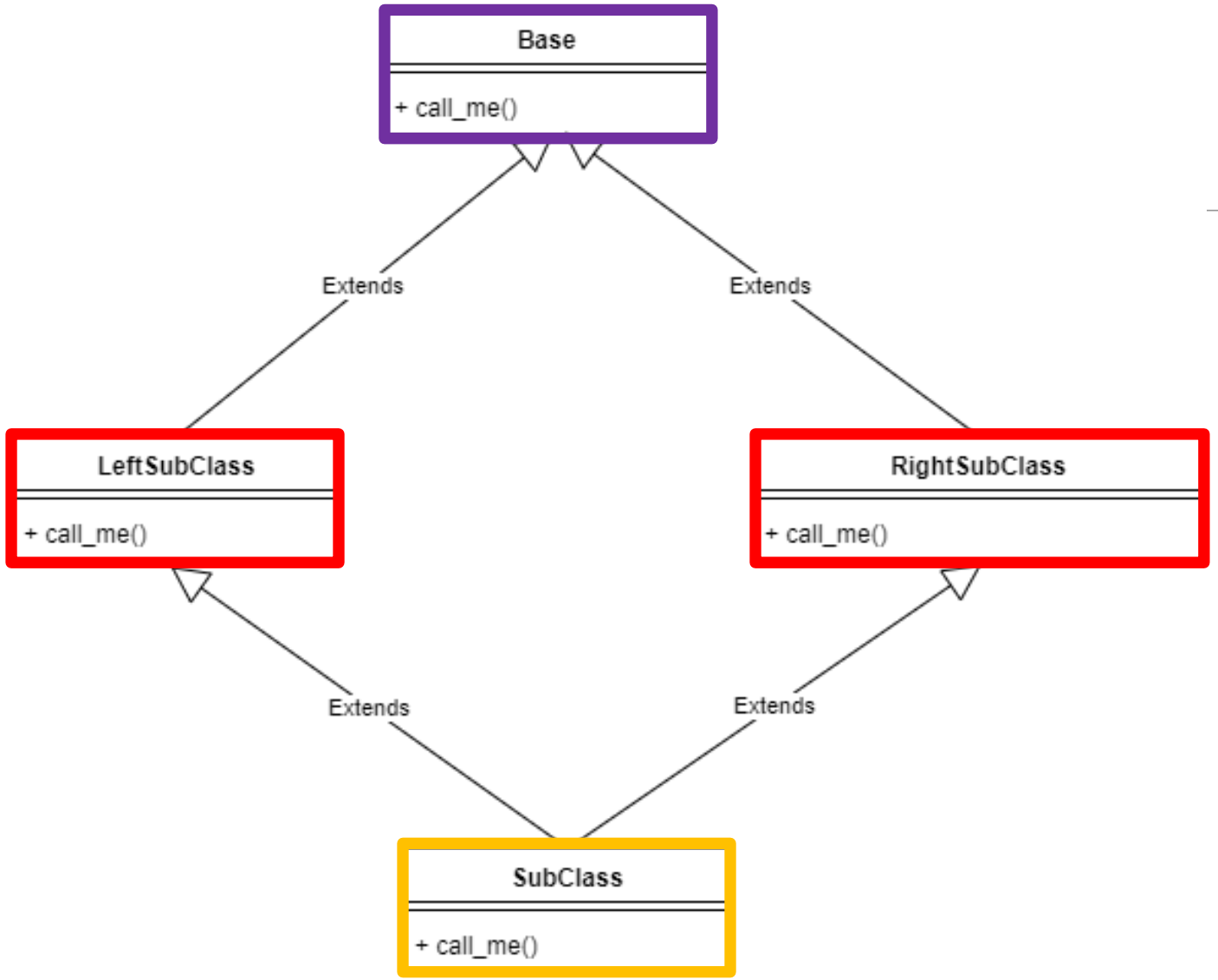


The Diamond Problem

SubClass inherits from **LeftSubClass**, which means it also inherits from **Base**

SubClass inherits from **RightSubClass**, which means it also inherits from a **second Base**?

Does that mean **SubClass** has two instances of **Base** as grandparents?



The Diamond Problem

One of the trickiest aspects of Multiple Inheritance is to determine the order in which methods are executed

All of the classes have a call_me() method

If **SubClass** invokes call_me() on its parent, **which call_me() function is called?**

- Is call_me() in **LeftSubClass** called?
- Is call_me() in **RightSubClass** called?
- Does it call both?
- Is call_me() in grandparent **Base** called?

The Diamond Problem

If call_me() method of a **SubClass** is invoked, in what order is the call_me() methods of its super classes invoked?

Let's take a look at some code!

multipleinheritance_diamond_problem.py

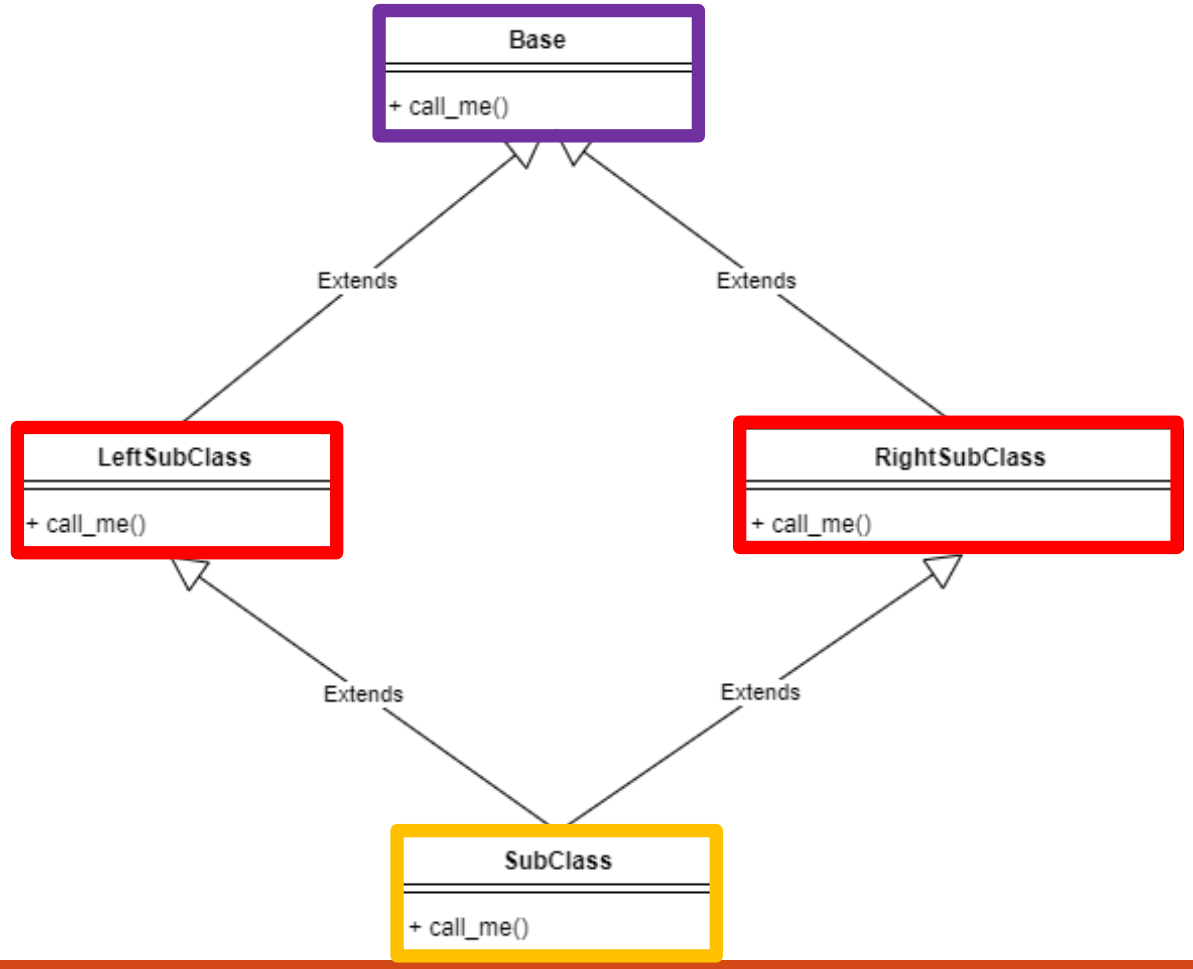
Notice in our code how it appears as though there are two Base grandparent classes if we call the parent's call_me() method directly?

We still don't know which parent's call_me() function is called

The Diamond Problem

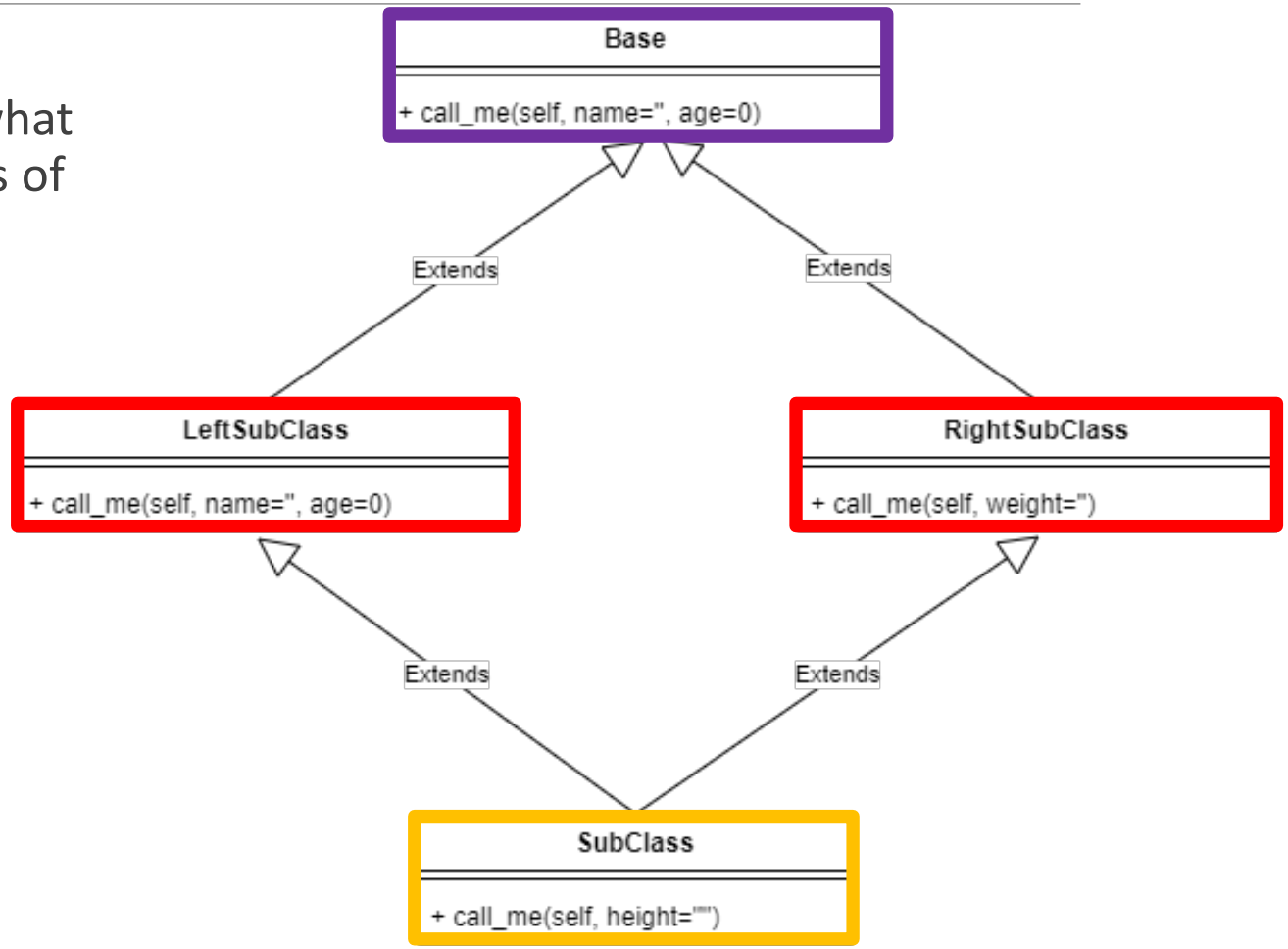
The **super()** keyword executes the methods in the parent class by first resolving the **Method Resolution Order**!

multipleinheritance_diamond_solution.py



The Diamond Problem

Great we figured out how to get around the problem of which method is called first. But what if all the call_me() methods took different sets of arguments?

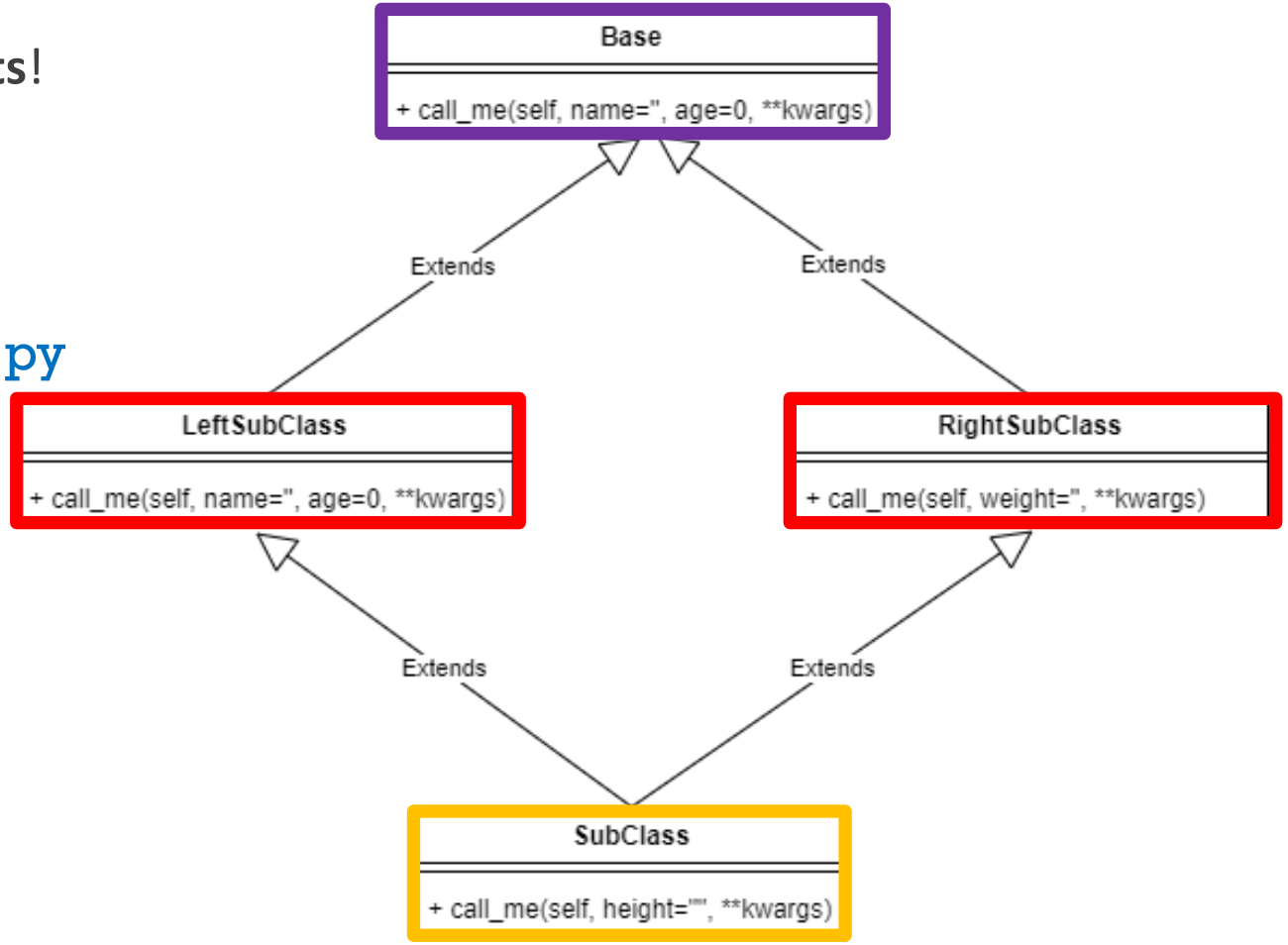


The Diamond Problem

<u>The Answer</u>: Variable Keyword Argument Lists!

We use the power of method overloading in python to pass a dictionary of a number of different argument types.

multipleinheritance_super_different_arguments.py



Interface Segregation Principle

Interface Segregation Principle

Multiple Inheritance is a great way to implement different Interfaces from parent Abstract Base Classes (ABC's)

Some rules to guide you:

- Make sure the interfaces don't overlap
- Avoid putting implementation in the ABC's unless absolutely necessary. **Emulate Java interfaces**. Treat these as pure abstractions.

```
class Walkable(abc.ABC):

    @abc.abstractmethod
    def walk(self):
        pass

class Swimmable(abc.ABC)

    @abc.abstractmethod
    def swim(self):
        pass

class Human(Walkable, Swimmable):

    def walk(self):
        # overridden walking code

    def swim(self):
        # overridden swimming code
```

Multiple Inheritance Alternatives

Alternatives?

Questions to ask yourself:

- Do I need inheritance? Will this code be re-used a lot? If not, I can just define it separately in the child class.
- Can I restructure my base classes?
- Can I **use composition instead**?